

REST API Assignment:

1. Define RESTful API principles and discuss their importance in web application development.

1. What is a RESTful API?

A RESTful API (Representational State Transfer API) is a way for different software systems to communicate over the internet using standard HTTP methods. It follows a set of architectural principles that make APIs simple, scalable, and easy to use.

- REST = Architectural style
- API = Interface for communication between systems

Example:

When you open a weather app, it requests data from a server using a REST API.

2. Core RESTful Principles

2.1 Stateless Communication

Each request from a client must contain all the information needed to process it.

- Server does not store client state
- Every request is independent
- Example:

GET /users/1

The server does not remember previous requests—everything needed is in this request.

2.2 Client-Server Architecture

Client and server are separated.

- Client → UI (browser, mobile app)
- Server → Data & logic

Benefit:

You can update UI without changing backend, and vice versa.

2.3 Uniform Interface

A consistent way to interact with resources using standard HTTP methods.

Method	Purpose
--------	---------

GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Remove data

Example:

GET /products → Get all products
POST /products → Add new product

2.4 Resource-Based Structure

Everything is treated as a resource (data object).

- **Resources are identified using URLs**

Example:

/users
/users/101
/orders/5001

2.5 Representation of Resources

Data is usually transferred in formats like:

- **JSON (most common)**
- **XML**

Example (JSON response):

```
{  
  "id": 1,  
  "name": "John"  
}
```

2.6 Stateless Caching

Responses can be cached to improve performance.

Example:

Browser stores API response → reduces repeated server calls.

2.7 Layered System

The system can have multiple layers (security, load balancer, etc.), but the client doesn't need to know.

3. Importance of RESTful APIs in Web Development

3.1 Scalability

- Stateless design allows handling millions of requests easily.
- Servers can be scaled horizontally.

3.2 Flexibility & Platform Independence

- Works with any frontend: React, Angular, mobile apps, etc.
- Language-independent (Java, Python, Node.js, etc.)

3.3 Simplicity

- Uses standard HTTP methods → easy to learn and implement.

3.4 Faster Development

- Frontend and backend teams can work independently.

3.5 Performance Optimization

- Caching reduces load time and improves speed.

3.6 Security

- Works with HTTPS, authentication (JWT, OAuth), etc.

4. Practical Real-World Example

E-commerce Website

Action	API Call
View products	GET /products
Add product	POST /products

Update product PUT /products/1

Delete product DELETE /products/1

2. Discuss the importance of asynchronous programming in Node.js. Explain the concepts of callbacks, promises, and async/await.

Why Asynchronous Programming is Important in Node.js

Node.js is built on a single-threaded, non-blocking event loop, which means it can handle many tasks at the same time without creating multiple threads.

• Problem with Synchronous Code

In synchronous execution:

- **Tasks run one after another**
- **If one task takes time (e.g., database call), everything else waits**

👉 Example:

```
const data = readFileSync("file.txt");  
console.log(data);  
console.log("Next task");
```

If file reading is slow → entire app is blocked

Solution: Asynchronous Programming

In asynchronous execution:

- **Long tasks run in the background**
- **Other code continues executing**

Example:

```
readFile("file.txt", (err, data) => {  
  console.log(data);
```

```
});  
console.log("Next task");
```

Output:

**Next task
(file content later)**

Why It Matters in Node.js

1. High Performance

Handles thousands of requests without blocking.

2. Scalability

Ideal for real-time apps (chat apps, APIs, streaming).

3. Better User Experience

No freezing or delays in execution.

2. Callbacks

What is a Callback?

A callback is a function passed as an argument to another function, executed later.

Example:

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data received");  
  }, 1000);  
}
```

```
fetchData((result) => {  
  console.log(result);  
});
```

Problem: Callback Hell

When callbacks are nested deeply → code becomes hard to read.

```
login(user, () => {
```

```
getProfile() => {  
  getPosts() => {  
    // messy code  
  };  
};  
};
```

Hard to maintain
Difficult debugging

3. Promises

What is a Promise?

A Promise represents a value that may be available now, later, or never.

States of Promise

- Pending
- Resolved (fulfilled)
- Rejected

Example:

```
const fetchData = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve("Data received");  
  }, 1000);  
});
```

fetchData

```
.then((data) => console.log(data))  
.catch((err) => console.log(err));
```

Advantages

- ✓ Avoids callback hell
- ✓ Cleaner chaining with **.then()**
- ✓ Better error handling

4. Async/Await

What is Async/Await?

A modern way to write asynchronous code that looks like synchronous code.

- **async** → declares a function
- **await** → waits for a promise to resolve

Example:

```
function fetchData() {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve("Data received"), 1000);  
  });  
}
```

```
async function getData() {  
  const data = await fetchData();  
  console.log(data);  
}
```

```
getData();
```

Error Handling:

```
async function getData() {  
  try {  
    const data = await fetchData();  
    console.log(data);  
  } catch (err) {  
    console.log(err);  
  }  
}
```

Advantages

- ✓ Cleaner and readable code
- ✓ Looks like synchronous logic
- ✓ Easier debugging

5. Comparison Summary

Feature	Callback	Promise	Async/Await
Readability	Low	Medium	High
Error Handle	Difficult	Better (.catch)	Best (try/catch)
Complexity	High	Moderate	Low

6. Real-World Example (API Call)

Using Promise:

```
fetch("https://api.example.com/data")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log(err));
```

Using Async/Await:

```
async function getData() {
  const res = await fetch("https://api.example.com/data");
  const data = await res.json();
  console.log(data);
}
```